UTN **FRT**	***UNIVERSIDAD TECNOLÓGICA NACIONAL*** *Facultad Regional Tucumán* Departamento: SISTEMAS Cátedra: Sintaxis y Semántica de los Lenguajes	**TRABAJO PRÁCTICO N° 3**

3. Ejercicio de Aplicación

Intérprete de Expresiones Algebraicas

Transductor de Pila (PDT) y Analizador SLR(1) con capacidad de cómputo

Alumno: _____

Legajo: _____

Comisión: _____

Docente: Ing. Jorge Buabud

Año: 2026

Interfaz web: `https://gagaismo.github.io/LaboratorioSSL2/`

Índice

1. Enunciado
2. Marco teórico: Transductor de Pila y Analizador LR
3. Problema 3.1: PDT para convertir expresiones infijas a posfijas
4. Problema 3.2: Evaluador de expresiones posfijas
5. Interfaz web de evaluación y modelos JFLAP
6. Conclusiones
7. Anexo A: Código fuente de la implementación

1. Enunciado

Podemos decir que una MEF es un AFD al que se le agregó una función de salida. De igual modo se puede definir un Transductor de Pila (Push Down Transducer, PDT) como un APD al que se le agrega una función de salida, definida sobre un alfabeto de salida Σ_S, con el mismo dominio que la función de transición del APD y rango en Σ_S^*:

$$\mathbf{g : (Q - F) \times (\Sigma_E \cup \{\Delta, \lambda\}) \times \Sigma_P^* \rightarrow \Sigma_S^*}$$

3.1. Aplicar el concepto de PDT para diseñar un modelo capaz de hacer el análisis sintáctico o reconocimiento de una expresión aritmética de sumas, productos, divisiones y restas, con paréntesis, en notación infija y convertirla a notación posfija. Suponer como operandos identificadores de variables o constantes representados por una letra del alfabeto inglés.

3.2. Implementar un algoritmo que use un modelo similar al APD para evaluar una expresión posfija, dados los valores numéricos de sus identificadores de variable o constante. Cuando lea operandos los introduce en la pila; cuando lea un operador saca los operandos necesarios de la pila, hace la operación y coloca el resultado en la pila, hasta que al terminar de leer la expresión posfija tenga el resultado como único valor en la pila.

2. Marco teórico

2.1. Transductor de Pila (PDT)

Un PDT es un APD al que se le agrega una cinta infinita de salida, donde puede escribir secuencias de un alfabeto de salida Σ_S. Partiendo del estado inicial con la cinta de salida vacía, en cada paso: (1) LEE (READ) un símbolo de la cinta de entrada; (2) DESCARGA (POP) una secuencia de la pila; (3) de acuerdo al estado actual, al símbolo leído y a la secuencia sacada de la pila, CARGA (PUSH) una secuencia en la pila, ESCRIBE (PRINT) una secuencia en la cinta de salida y transiciona a un ESTADO NUEVO; (4) repite hasta llegar a un estado final de parada y aceptación o a un bloqueo con rechazo. Su definición formal es:

$$\mathbf{PDT = \langle\ Q, \Sigma_E, \Sigma_S, \Sigma_P, q_0, z_0, F, f, g\ \rangle}$$

con $f : (Q - F) \times (\Sigma_E \cup \{\Delta, \lambda\}) \times \Sigma_P^* \rightarrow Q \times \Sigma_P^*$ y $g : (Q - F) \times (\Sigma_E \cup \{\Delta, \lambda\}) \times \Sigma_P^* \rightarrow \Sigma_S^*$. Por ser determinista, no se permiten transiciones con READ y POP simultáneamente nulos, ni ninguna situación que produzca más de una reacción para un mismo estímulo.

2.2. Analizador LR

Los analizadores LR(K) son autómatas de pila diseñados a partir de una GIC, con análisis ascendente por derivaciones más a la derecha, que recorren la entrada desde la izquierda y usan preanálisis de K símbolos. El caso más simple, con K = 1, es el SLR(1). Se construye un AF a partir de las reglas de la GIC; sus estados son los *símbolos especiales* que se cargan en la pila del AP junto con los símbolos gramaticales, y a partir de ese AF se obtiene la Tabla de Análisis LR (en este trabajo, con JFLAP). Las acciones de cómputo se realizan junto con las reducciones.

3. Problema 3.1: PDT para convertir expresiones infijas a posfijas

3.1. Consideraciones de diseño

- Se toma como base el PDT analizador-conversor de la cátedra (estados q0 a q6, construido en JFLAP para los operadores + y *) y se lo extiende a los cuatro operadores pedidos: +, −, * y /.
- Como JFLAP reserva los paréntesis, la expresión se escribe con corchetes `[ ]`. La interfaz acepta también paréntesis y los reemplaza por corchetes antes del análisis.
- El fin de la cadena de entrada se representa con el símbolo ⌂ (Δ en JFLAP) y el símbolo inicial de la pila es `Z`.
- Los operandos son letras minúsculas del alfabeto inglés (a..z). En las tablas se agrupan bajo el nombre `letra`.
- Precedencia: * y / tienen mayor precedencia que + y −; a igual precedencia se asocia a izquierda.
- A efectos de la implementación, en lugar del bloqueo del modelo se usa un estado de rechazo tipo sumidero, adonde se dirigen todas las transiciones no definidas. Como q7 y q8 se usan para los nuevos operadores, el sumidero es **q9**.

Idea del modelo: los operandos se escriben inmediatamente en la salida; el corchete de apertura se apila; un operador no puede escribirse hasta que haya salido su segundo operando, por eso se apila, pero antes deben salir de la pila (y escribirse) los operadores de igual o mayor precedencia que estén en el tope. El corchete de cierre vacía la pila hasta el corchete de apertura, y el fin de cadena vacía la pila hasta `Z`.

Las pilas tienen siempre una forma muy acotada entre dos corchetes: a lo sumo un operador de baja precedencia y, encima, uno de alta. Al leer un operador hay que recordar *cuál* se leyó mientras se vacía la pila, para apilarlo al final. En el modelo de la cátedra eso lo hacen q1 (se leyó *) y q2 (se leyó +); con cuatro operadores se agregan q7 (se leyó /) y q8 (se leyó −).

3.2. Definición formal

$$\mathbf{PDT = \langle\ Q,\ \Sigma_E,\ \Sigma_S,\ \Sigma_P,\ q_0,\ z_0,\ F,\ f,\ g\ \rangle}$$

- $Q = \{ q0, q1, q2, q3, q4, q5, q6, q7, q8 \}$
- $\Sigma_E = \{ a, b, \ldots, z, +, -, *, /, [,] \}$
- $\Sigma_S = \{ a, b, \ldots, z, +, -, *, / \}$
- $\Sigma_P = \{ Z, [, +, -, *, / \}$
- $q_0 = q0 \quad z_0 = Z \quad F = \{ q6 \}$
- f y g: definidas por el grafo de la figura 1 y la tabla de transición-salida de la sección 3.4.

Estado	Función	READ	POP
q0	Inicial. Espera un operando o un corchete de apertura.	sí	no
q1	Se leyó *: saca los operadores * y / del tope y luego apila *.	no	sí
q2	Se leyó +: saca todos los operadores hasta [o Z y luego apila +.	no	sí
q3	Se leyó un operando. Espera un operador, un corchete de cierre o el fin de cadena.	sí	sí
q4	Se leyó el fin de cadena: vacía la pila hasta Z.	no	sí
q5	Se leyó un corchete de cierre: vacía la pila hasta [.	no	sí
q6	Final de parada y aceptación.	—	—
q7	Se leyó /: saca los operadores * y / del tope y luego apila /.	no	sí

q8	Se leyó −: saca todos los operadores hasta [o Z y luego apila −.	no	sí
q9	Sumidero de rechazo (solo en la implementación).	—	—

3.3. Grafo de transición

Para que el grafo sea legible, las transiciones análogas se agrupan con las siguientes variables: `o` ∈ {+, −, *, /} (cualquier operador), `h` ∈ {*, /} (alta precedencia), `l` ∈ {+, −} (baja precedencia) y `letra` ∈ {a, …, z}. Cada arista tiene la forma `READ , POP ; PUSH` y la salida se indica con `Print`, como en JFLAP.

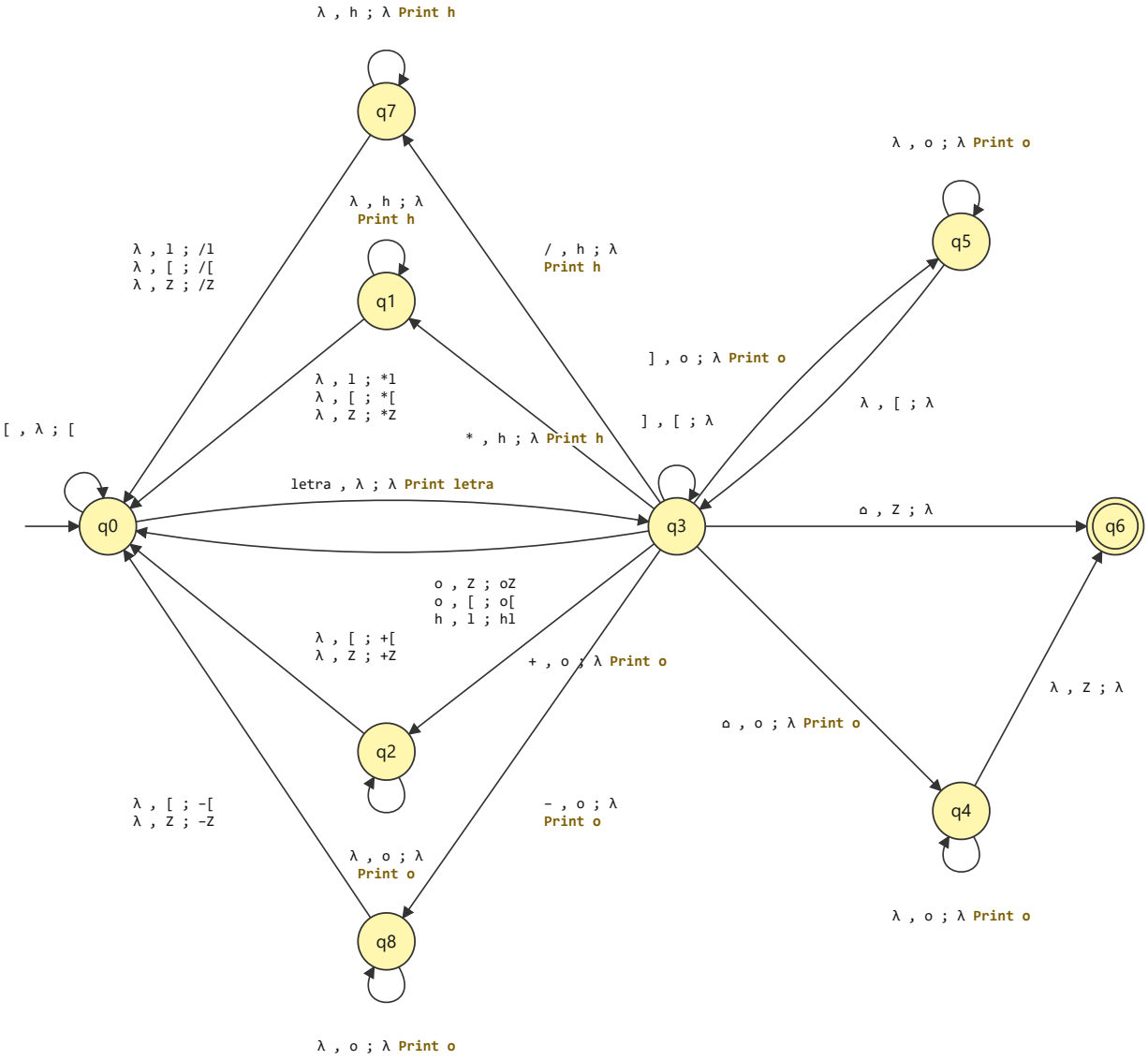


Figura 1. Grafo de transición del PDT analizador-conversor de expresiones infijas a posfijas.

3.4. Tabla de transición-salida

Cada fila es una transición válida. Los casilleros λ representan la No-operación. Toda combinación de estímulo (estado actual, READ, POP) que no figura en la tabla lleva al sumidero q9 (rechazo).

N°	Estado actual	READ	POP	Estado nuevo	PUSH	PRINT
0	→ q0	letra	λ	q3	λ	letra
1	q0	[	λ	q0	[	λ
2	q1	λ	Z	q0	*Z	λ
3	q1	λ	+	q0	*+	λ
4	q1	λ	-	q0	*-	λ
5	q1	λ	*	q1	λ	*
6	q1	λ	/	q1	λ	/
7	q1	λ	[	q0	*[	λ
8	q2	λ	Z	q0	+Z	λ
9	q2	λ	+	q2	λ	+
10	q2	λ	-	q2	λ	-
11	q2	λ	*	q2	λ	*
12	q2	λ	/	q2	λ	/
13	q2	λ	[	q0	+[	λ
14	q3	+	Z	q0	+Z	λ
15	q3	+	[	q0	+[	λ
16	q3	+	+	q2	λ	+
17	q3	+	-	q2	λ	-
18	q3	+	*	q2	λ	*
19	q3	+	/	q2	λ	/
20	q3	-	Z	q0	-Z	λ
21	q3	-	[	q0	-[	λ
22	q3	-	+	q8	λ	+
23	q3	-	-	q8	λ	-
24	q3	-	*	q8	λ	*
25	q3	-	/	q8	λ	/
26	q3	*	Z	q0	*Z	λ
27	q3	*	[	q0	*[	λ
28	q3	*	+	q0	*+	λ
29	q3	*	-	q0	*-	λ
30	q3	*	*	q1	λ	*
31	q3	*	/	q1	λ	/
32	q3	/	Z	q0	/Z	λ
33	q3	/	[	q0	/[	λ
34	q3	/	+	q0	/+	λ
35	q3	/	-	q0	/-	λ

36	q3	/	*	q7	λ	*
37	q3	/	/	q7	λ	/
38	q3	]	+	q5	λ	+
39	q3	]	-	q5	λ	-
40	q3	]	*	q5	λ	*
41	q3	]	/	q5	λ	/
42	q3	]	[	q3	λ	λ
43	q3	⌂	Z	* q6	λ	λ
44	q3	⌂	+	q4	λ	+
45	q3	⌂	-	q4	λ	-
46	q3	⌂	*	q4	λ	*
47	q3	⌂	/	q4	λ	/
48	q4	λ	+	q4	λ	+
49	q4	λ	-	q4	λ	-
50	q4	λ	*	q4	λ	*
51	q4	λ	/	q4	λ	/
52	q4	λ	Z	* q6	λ	λ
53	q5	λ	+	q5	λ	+
54	q5	λ	-	q5	λ	-
55	q5	λ	*	q5	λ	*
56	q5	λ	/	q5	λ	/
57	q5	λ	[	q3	λ	λ
58	q7	λ	Z	q0	/Z	λ
59	q7	λ	+	q0	/+	λ
60	q7	λ	-	q0	/-	λ
61	q7	λ	*	q7	λ	*
62	q7	λ	/	q7	λ	/
63	q7	λ	[	q0	/[	λ
64	q8	λ	Z	q0	-Z	λ
65	q8	λ	+	q8	λ	+
66	q8	λ	-	q8	λ	-
67	q8	λ	*	q8	λ	*
68	q8	λ	/	q8	λ	/
69	q8	λ	[	q0	-[	λ

3.5. Implementación

Se implementaron las dos estrategias propuestas por la cátedra. Ambas usan el mismo algoritmo de análisis y conversión; solo cambia cómo se obtiene la reacción del PDT a partir del estímulo (estado actual, READ, POP). Según el estado se decide si se lee la entrada y si se desapila: en q0 se lee y no se desapila; en q1, q2, q4, q5, q7 y q8 no se lee (λ) y se desapila; en q3 se lee y se desapila.

Primera estrategia: tabla como arreglo bidimensional. La tabla de la sección 3.4 se guarda como una matriz donde cada fila es una transición. El algoritmo busca secuencialmente la fila que corresponde al estímulo; si la búsqueda fracasa, se rechaza la secuencia.

```
Estado=q0
mientras (estado != q6 and estado != q9)
  caso estado
                    q0: leer(símb_entrada)
                        símb_pila=vacío
    q1, q2, q4, q5, q7, q8: símb_entrada=vacío
                        pop(símb_pila)
                    q3: leer(símb_entrada)
                        pop(simb_pila)
  fin caso
  pos=buscarEnTabla(estado, símb_entrada, simb_pila)
  si (pos != q9)
    push(tabla(pos,sec_push))
    print(tabla(pos,print))
    estado=tabla(pos,nuevo_estado)
  sino
    estado=q9
  fin si
fin mientras
si (estado == q6)
  entonces rutina_aceptación()
  sino rutina_rechazo()
fin si
```

Segunda estrategia: tabla como arreglo tridimensional. Los símbolos de entrada y de pila y los estados se convierten en índices numéricos, para acceder directamente a cada elemento de la tabla mediante el estímulo `tabla[estado][read][pop]`. Cada elemento es un registro con tres campos: ESTADO NUEVO (entero), PUSH (cadena) y PRINT (cadena). Los casilleros no definidos contienen el estado 9 (sumidero). La letra de entrada y el símbolo Z de la pila comparten el índice 1, porque nunca aparecen en el mismo eje.

Símbolo	Índice
λ	0
letra a..z , Z	1
+	2
−	3
*	4
/	5
[	6
]	7
⌂	8

Estado	Índice
q0 … q8	0 … 8
q9 (sumidero)	9

```
Estado=0
mientras (estado != 6 and estado != 9)
  caso estado
                0: leer(símb_entrada)
                   símb_pila=vacío
    1, 2, 4, 5, 7, 8: símb_entrada=vacío
                   pop(símb_pila)
                3: leer(símb_entrada)
                   pop(simb_pila)
  fin caso
  push(tabla(estado,símb_entrada,simb_pila).sec_push)
  print(tabla(estado,símb_entrada,simb_pila).sec_print)
  estado=tabla(estado,símb_entrada,simb_pila).nuevo_estado
fin mientras
si (estado == 6)
  entonces rutina_aceptación()
  sino rutina_rechazo()
fin si
```

La tabla tridimensional se carga a partir de las mismas filas de la sección 3.4, expresadas con índices:

Estado actual	READ	POP	Estado nuevo	PUSH	PRINT
0	1	0	3		letra
0	6	0	0	[	
1	0	1	0	*Z	
1	0	2	0	*+	
1	0	3	0	*-	
1	0	4	1		*
1	0	5	1		/
1	0	6	0	*[	
2	0	1	0	+Z	
2	0	2	2		+
2	0	3	2		-
2	0	4	2		*
2	0	5	2		/
2	0	6	0	+[	
3	2	1	0	+Z	
3	2	6	0	+[	
3	2	2	2		+
3	2	3	2		-
3	2	4	2		*
3	2	5	2		/
3	3	1	0	-Z	
3	3	6	0	-[	
3	3	2	8		+
3	3	3	8		-
3	3	4	8		*
3	3	5	8		/
3	4	1	0	*Z	
3	4	6	0	*[	
3	4	2	0	*+	
3	4	3	0	*-	
3	4	4	1		*
3	4	5	1		/
3	5	1	0	/Z	
3	5	6	0	/[	
3	5	2	0	/+	
3	5	3	0	/-	
3	5	4	7		*
3	5	5	7		/
3	7	2	5		+
3	7	3	5		-
3	7	4	5		*
3	7	5	5		/
3	7	6	3		
3	8	1	6		
3	8	2	4		+
3	8	3	4		-
3	8	4	4		*
3	8	5	4		/
4	0	2	4		+
4	0	3	4		-

4	0	4	4		*
4	0	5	4		/
4	0	1	6		
5	0	2	5		+
5	0	3	5		-
5	0	4	5		*
5	0	5	5		/
5	0	6	3		
7	0	1	0	/Z	
7	0	2	0	/+	

7	0	3	0	/-	
7	0	4	7		*
7	0	5	7		/
7	0	6	0	/[	
8	0	1	0	-Z	
8	0	2	8		+
8	0	3	8		-
8	0	4	8		*
8	0	5	8		/
8	0	6	0	-[	

La implementación (función `convertirInfijaAPosfija` del Anexo A) se verificó con 3000 expresiones generadas al azar: ambas estrategias producen exactamente las mismas trazas y la misma expresión posfija que el algoritmo clásico de conversión, y rechazan las cadenas mal formadas (operadores consecutivos, corchetes desbalanceados, operandos contiguos, símbolos fuera del alfabeto, cadena vacía).

3.6. Trazas

Ejemplo de aceptación. Entrada `[a+b]*c-d/e␣`. La pila se muestra con el tope a la izquierda, igual que las secuencias de PUSH.

Paso	Estado actual	READ	POP	Estado nuevo	PUSH	PRINT	Pila (tope a la izq.)	Cinta de salida
1	q0	[	λ	q0	[	λ	[Z	λ
2	q0	a	λ	q3	λ	a	[Z	a
3	q3	+	[	q0	+[	λ	+[Z	a
4	q0	b	λ	q3	λ	b	+[Z	ab
5	q3	]	+	q5	λ	+	[Z	ab+
6	q5	λ	[	q3	λ	λ	Z	ab+
7	q3	*	Z	q0	*Z	λ	*Z	ab+
8	q0	c	λ	q3	λ	c	*Z	ab+c
9	q3	-	*	q8	λ	*	Z	ab+c*
10	q8	λ	Z	q0	-Z	λ	-Z	ab+c*
11	q0	d	λ	q3	λ	d	-Z	ab+c*d
12	q3	/	-	q0	/-	λ	/-Z	ab+c*d
13	q0	e	λ	q3	λ	e	/-Z	ab+c*de
14	q3	␣	/	q4	λ	/	-Z	ab+c*de/
15	q4	λ	-	q4	λ	-	Z	ab+c*de/-
16	q4	λ	Z	q6	λ	λ	λ	ab+c*de/-

La cadena es aceptada en q6 y la cinta de salida contiene la expresión posfija **`ab+c*de/-`**.

Ejemplo de rechazo. Entrada `a+*b␣`: después de leer + el PDT queda en q0 esperando un operando, y no hay transición definida para (q0, *, λ), por lo que pasa al sumidero q9.

Paso	Estado actual	READ	POP	Estado nuevo	PUSH	PRINT	Pila (tope a la izq.)	Cinta de salida
1	q0	a	λ	q3	λ	a	Z	a
2	q3	+	Z	q0	+Z	λ	+Z	a
3	**q0**	*****	**λ**	**q9**	**λ**	**λ**	**+Z**	**a**

4. Problema 3.2: Evaluador de expresiones posfijas

4.1. Gramática Independiente del Contexto

El evaluador se basa en un Analizador SLR(1) construido con JFLAP a partir de la GIC de las expresiones posfijas. Se extiende la gramática de la cátedra (S → SS+ | SS* | a) a los cuatro operadores; el terminal `a` representa cualquier operando:

Regla	Producción
r1	`S → SS+`
r2	`S → SS−`
r3	`S → SS*`
r4	`S → SS/`
r5	`S → a`

Paso 1) Gramática ampliada: se agrega la regla `S' → S`. Σ_T = { a, +, −, *, / } Σ_N = { S', S }.

4.2. Conjuntos PRIMERO y SIGUIENTE

Paso 2)

	PRIMERO	SIGUIENTE
S	{ a }	{ *, +, −, /, a, $ }

SIGUIENTE(S) contiene `$` por S' → S; contiene los cuatro operadores porque S aparece antes del operador en S → SS*op*; y contiene `a` porque el primer S de SS*op* está seguido por otro S, cuyo PRIMERO es { a }.

4.3. Autómata finito de elementos LR(0)

Paso 3) Estados del AF (símbolos especiales) con sus elementos y transiciones:

Estado	Elementos	Transiciones

0	`S' → ·S`
	`S → ·SS+ | ·SS− | ·SS* | ·SS/`
	`S → ·a`
		`S → 1`
		`a → 2`

1	`S' → S·`
	`S → S·S+ | S·S− | S·S* | S·S/`
	`S → ·SS+ | ·SS− | ·SS* | ·SS/`
	`S → ·a`
		`S → 3`
		`a → 2`

2	`S → a·`	—

3	`S → SS·+ | SS·− | SS·* | SS·/`
	`S → S·S+ | S·S− | S·S* | S·S/`
	`S → ·SS+ | ·SS− | ·SS* | ·SS/`
	`S → ·a`
		`* → 4  + → 5`
		`− → 6  / → 7`
		`S → 3  a → 2`

4	`S → SS*·`	—
5	`S → SS+·`	—
6	`S → SS−·`	—
7	`S → SS/·`	—

4.4. Tabla de análisis SLR(1)

Paso 4) La tabla se representa con un arreglo bidimensional: el primer índice es el símbolo especial (estado del AF) y el segundo los símbolos terminales, el no terminal S y el símbolo $ de fin de cadena. `sk` indica desplazar e ir al estado k; `rj`, reducir por la regla j; `acc`, aceptar; el casillero vacío, error. La tabla no tiene conflictos, por lo que la gramática es SLR(1).

Estado	ACCIÓN						IR_A
	*	+	-	/	a	$	S
0					s2		1
1					s2	acc	3
2	r5	r5	r5	r5	r5	r5	
3	s4	s5	s6	s7	s2		3
4	r3	r3	r3	r3	r3	r3	
5	r1	r1	r1	r1	r1	r1	
6	r2	r2	r2	r2	r2	r2	
7	r4	r4	r4	r4	r4	r4	

4.5. Estructuras de datos

Se utilizan dos pilas:

1. **Pila de análisis:** la del AP que realiza el análisis sintáctico; guarda los símbolos especiales alternados con los símbolos gramaticales.
2. **Pila de cálculo:** guarda los valores numéricos, tal como indica el enunciado.

Y dos arreglos unidimensionales, construidos a partir de la expresión posfija:

1. Uno de tipo carácter, con la expresión posfija simbólica: cada operando se reemplaza por `a`, se eliminan los espacios en blanco y se agrega `$` al final. Es la entrada del AP.
2. Otro numérico, con los valores de los operandos en el orden en que aparecen. Sirve para reemplazar cada `a` por su valor al momento del cálculo.

Los operandos pueden darse como identificadores (letras, cuyo valor se ingresa por separado, como pide el enunciado) o directamente como números separados por espacios, como en las pautas de la cátedra (por ejemplo `10 4 + 2 3 * +`).

4.6. Algoritmo

```
CargarTabla()
Simb_especial = 0
insertar(Simb_especial)
leer(simbolo)
valor_tabla = tabla[Simb_especial, simbolo]
mientras (valor_tabla != aceptar)
  caso (valor_tabla sea)
    desplazamiento:
      insertar(simbolo)
      Simb_especial = valor_tabla.estado
      insertar(Simb_especial)
      leer(simbolo)
    reducción:
      extraer(lado_derecho_de_valor_tabla.ReglaReescritura)
      Simb_especial = elemento_cima_de_la_pila_sin_extraer
      insertar(lado_izquierdo_de_valor_tabla.ReglaReescritura)
      Simb_especial = tabla[Simb_especial, lado_izquierdo_de_valor_tabla.ReglaReescritura]
      insertar(Simb_especial)
      operar_en_pila_de_calculo(lado_derecho_de_valor_tabla.ReglaReescritura)
    vacío:
      salir a la rutina_de_error()
  fin_caso
  valor_tabla = tabla[Simb_especial, simbolo]
fin_mientras
si (simbolo no es $)
  salir a la rutina_de_error()
fin_si
vaciar_pila()
mostrar_resultado()
```

Al reducir se extraen de la pila de análisis 2·|lado derecho| elementos (cada símbolo gramatical con su símbolo especial). La acción `operar_en_pila_de_calculo` es la que realiza el cómputo, junto con la reducción:

- **r5 (S → a):** se toma el siguiente valor del arreglo numérico y se inserta en la pila de cálculo.
- **r1 a r4 (S → SS*op*):** se extrae val2 y luego val1 de la pila de cálculo, se calcula val1 *op* val2 y se inserta el resultado. Si el operador es / y val2 es 0, se informa división por cero.

Al aceptar, el único valor que queda en la pila de cálculo es el resultado de la expresión.

4.7. Traza

Se evalúa la expresión posfija obtenida en el problema 3.1, `ab+c*de/-`, con los valores a = 2, b = 3, c = 4, d = 10, e = 5. Arreglo simbólico: `a a + a * a a / - $`; arreglo numérico: `2 3 4 10 5`. Equivale a [2+3]*4 − 10/5 = 18. Cada fila muestra la configuración (pilas y entrada restante) antes de aplicar la acción indicada.

Paso	Pila de análisis	Entrada	Acción	Pila de cálculo
1	`0`	`aa+a*aa/-$`	`Desplazar s2`	`vacía`
2	`0 a 2`	`a+a*aa/-$`	`Reducir r5: S → a [apila 2]`	`vacía`
3	`0 S 1`	`a+a*aa/-$`	`Desplazar s2`	`2`
4	`0 S 1 a 2`	`+a*aa/-$`	`Reducir r5: S → a [apila 3]`	`2`
5	`0 S 1 S 3`	`+a*aa/-$`	`Desplazar s5`	`2 3`
6	`0 S 1 S 3 + 5`	`a*aa/-$`	`Reducir r1: S → SS+ [2 + 3 = 5]`	`2 3`
7	`0 S 1`	`a*aa/-$`	`Desplazar s2`	`5`
8	`0 S 1 a 2`	`*aa/-$`	`Reducir r5: S → a [apila 4]`	`5`
9	`0 S 1 S 3`	`*aa/-$`	`Desplazar s4`	`5 4`
10	`0 S 1 S 3 * 4`	`aa/-$`	`Reducir r3: S → SS* [5 * 4 = 20]`	`5 4`
11	`0 S 1`	`aa/-$`	`Desplazar s2`	`20`
12	`0 S 1 a 2`	`a/-$`	`Reducir r5: S → a [apila 10]`	`20`
13	`0 S 1 S 3`	`a/-$`	`Desplazar s2`	`20 10`
14	`0 S 1 S 3 a 2`	`/-$`	`Reducir r5: S → a [apila 5]`	`20 10`
15	`0 S 1 S 3 S 3`	`/-$`	`Desplazar s7`	`20 10 5`
16	`0 S 1 S 3 S 3 / 7`	`-$`	`Reducir r4: S → SS/ [10 / 5 = 2]`	`20 10 5`
17	`0 S 1 S 3`	`-$`	`Desplazar s6`	`20 2`
18	`0 S 1 S 3 - 6`	`$`	`Reducir r2: S → SS- [20 - 2 = 18]`	`20 2`
19	`0 S 1`	`$`	`Aceptar`	`18`

Resultado: **18**.

5. Interfaz web de evaluación y modelos JFLAP

Los dos problemas pueden evaluarse en la página `https://gagaismo.github.io/LaboratorioSSL2/`, que ejecuta en el navegador la misma implementación del Anexo A:

- **Problema 3.1:** se ingresa una expresión infija, se elige la estrategia (arreglo bidimensional o tridimensional) y se muestra si es aceptada o rechazada, la expresión posfija y la traza completa del PDT. También se puede consultar la tabla de transición-salida.
- **Problema 3.2:** se ingresa una expresión posfija (o se trae la obtenida en 3.1), se cargan los valores de sus identificadores y se muestra el resultado con la traza de la pila de análisis y la pila de cálculo. También se puede consultar la tabla SLR(1).

En la carpeta `modelos/` de la página se incluyen los archivos de JFLAP: `PDT-Analizador-Conversor-ExpAlg-4op.jff` (el PDT modelado como autómata de pila con notas *Print*, con los operandos a, b y c como en el modelo de la cátedra) y `GICinterpretePosfijo-4op.jff` (la GIC, a partir de la cual JFLAP genera la tabla SLR(1) de la sección 4.4).

6. Conclusiones

Con un PDT determinista de nueve estados se resuelve el reconocimiento de expresiones infijas con los cuatro operadores y corchetes, junto con su traducción a notación posfija, respetando la precedencia y la asociatividad a izquierda. La extensión del modelo de la cátedra solo requirió agregar un estado de vaciado por cada nuevo operador, porque el PDT debe recordar qué operador leyó mientras saca de la pila los de igual o mayor precedencia. Las dos estrategias de implementación dan los mismos resultados; la tridimensional accede directamente a la reacción, sin búsqueda secuencial.

El evaluador del problema 3.2 muestra un analizador SLR(1) con capacidad de cómputo: la pila de análisis valida la sintaxis de la expresión posfija y las acciones semánticas asociadas a las reducciones operan sobre una segunda pila, de modo que al aceptar la cadena el resultado queda como único valor en la pila de cálculo.

Anexo A: Código fuente de la implementación

Archivo `tp3.js` (JavaScript). Contiene ambas estrategias del problema 3.1 y el evaluador del problema 3.2.

```
// TP Nº 3 - Ejercicio de Aplicación: Intérprete de Expresiones Algebraicas.
// Sintaxis y Semántica de los Lenguajes - UTN FRT.
// Problema 3.1: PDT que analiza una expresión infija y la convierte a posfija.
// Problema 3.2: Analizador SLR(1) con pila de cálculo que evalúa la expresión posfija.

(function (global) {
  'use strict';

  // ====================================================================
  // PROBLEMA 3.1 - TRANSDUCTOR DE PILA (PDT)
  // ====================================================================

  var LAMBDA = '';      // casillero en blanco = λ (No-operación)
  var FIN = 'ɑ';        // fin de cadena (símbolo de JFLAP)
  var ACEPTACION = 6;   // q6: estado final de parada y aceptación
  var SUMIDERO = 9;     // q9: estado de rechazo tipo sumidero

  var BAJA = ['+', '-'];
  var ALTA = ['*', '/'];
  var OPERADORES = ['+', '-', '*', '/'];

  // Estado de vaciado al que se pasa cuando el operador leído debe esperar
  // a que salgan de la pila los operadores de igual o mayor precedencia.
  var VACIADO = { '*': 1, '+': 2, '/': 7, '-': 8 };

  // Primera estrategia: tabla como arreglo bidimensional.
  // Cada fila: [ESTADO ACTUAL, READ, POP, ESTADO NUEVO, PUSH, PRINT]
  // "letra" representa cualquier letra minúscula del alfabeto inglés.
  function construirTablaBidimensional() {
    var t = [];
    function fila(e, read, pop, nuevo, push, print) { t.push([e, read, pop, nuevo, push, print]); }

    // q0: espera un operando o una apertura de corchete
    fila(0, 'letra', LAMBDA, 3, LAMBDA, 'letra');
    fila(0, '[', LAMBDA, 0, '[', LAMBDA);

    // q1 (leyó *), q2 (leyó +), q7 (leyó /), q8 (leyó -): vaciado sin leer entrada
    [1, 2].forEach(function (q) { filasVaciado(q); });

    // q3: después de un operando, lee y desapila
    OPERADORES.forEach(function (op) {
      fila(3, op, 'Z', 0, op + 'Z', LAMBDA);
      fila(3, op, '[', 0, op + '[', LAMBDA);
      OPERADORES.forEach(function (tope) {
        if (ALTA.indexOf(op) >= 0 && BAJA.indexOf(tope) >= 0) {
          fila(3, op, tope, 0, op + tope, LAMBDA);      // mayor precedencia: espera arriba
        } else {
          fila(3, op, tope, VACIADO[op], LAMBDA, tope); // sale el de igual o mayor precedencia
        }
      });
    });
    OPERADORES.forEach(function (tope) { fila(3, ']', tope, 5, LAMBDA, tope); });
    fila(3, ']', '[', 3, LAMBDA, LAMBDA);
    fila(3, FIN, 'Z', 6, LAMBDA, LAMBDA);
    OPERADORES.forEach(function (tope) { fila(3, FIN, tope, 4, LAMBDA, tope); });

    // q4: fin de cadena, vacía la pila hasta Z
    OPERADORES.forEach(function (tope) { fila(4, LAMBDA, tope, 4, LAMBDA, tope); });
    fila(4, LAMBDA, 'Z', 6, LAMBDA, LAMBDA);
```

```
  // q5: cierre de corchete, vacía la pila hasta [
  OPERADORES.forEach(function (tope) { fila(5, LAMBDA, tope, 5, LAMBDA, tope); });
  fila(5, LAMBDA, '[', 3, LAMBDA, LAMBDA);

  [7, 8].forEach(function (q) { filasVaciado(q); });

  function filasVaciado(q) {
    var leido = Object.keys(VACIADO).filter(function (op) { return VACIADO[op] === q; })[0];
    var esAlta = ALTA.indexOf(leido) >= 0;
    fila(q, LAMBDA, 'Z', 0, leido + 'Z', LAMBDA);
    OPERADORES.forEach(function (tope) {
      if (esAlta && BAJA.indexOf(tope) >= 0) fila(q, LAMBDA, tope, 0, leido + tope, LAMBDA);
      else fila(q, LAMBDA, tope, q, LAMBDA, tope);
    });
    fila(q, LAMBDA, '[', 0, leido + '[', LAMBDA);
  }
  return t;
}

var TABLA_PDT = construirTablaBidimensional();

function esLetra(c) { return c.length === 1 && c >= 'a' && c <= 'z'; }

function buscarEnTabla(estado, entrada, pila) {
  var read = esLetra(entrada) ? 'letra' : entrada;
  for (var pos = 0; pos < TABLA_PDT.length; pos++) {
    var f = TABLA_PDT[pos];
    if (f[0] === estado && f[1] === read && f[2] === pila) return pos;
  }
  return -1; // la búsqueda fracasa: se rechaza la secuencia
}

// Segunda estrategia: tabla como arreglo tridimensional tabla[estado][read][pop].
// Los símbolos de entrada y de pila se convierten en índices numéricos.
var INDICE_SIMBOLO = { '': 0, 'letra': 1, 'Z': 1, '+': 2, '-': 3, '*': 4, '/': 5, '[': 6, ']': 7 };
INDICE_SIMBOLO[FIN] = 8;
var CANT_SIMBOLOS = 9;
var CANT_ESTADOS = 10; // q0..q9

function construirTablaTridimensional() {
  var t = [];
  for (var e = 0; e < CANT_ESTADOS; e++) {
    t[e] = [];
    for (var r = 0; r < CANT_SIMBOLOS; r++) {
      t[e][r] = [];
      for (var p = 0; p < CANT_SIMBOLOS; p++) {
        t[e][r][p] = { nuevo_estado: SUMIDERO, sec_push: LAMBDA, sec_print: LAMBDA };
      }
    }
  }
  TABLA_PDT.forEach(function (f) {
    t[f[0]][INDICE_SIMBOLO[f[1]]][INDICE_SIMBOLO[f[2]]] =
      { nuevo_estado: f[3], sec_push: f[4], sec_print: f[5] };
  });
  return t;
}

var TABLA_PDT_3D = construirTablaTridimensional();

function indice(simbolo) {
  if (esLetra(simbolo)) return 1;
  return INDICE_SIMBOLO.hasOwnProperty(simbolo) ? INDICE_SIMBOLO[simbolo] : -1;
}

// Algoritmo de Análisis y Conversión (estrategia 1 o 2).
// La expresión usa corchetes como en JFLAP; el fin de cadena se agrega aquí.
function convertirInfijaAPosfija(expresion, estrategia) {
```

```
    var cinta = expresion.replace(/\s+/g, '') + FIN;
    var i = 0;
    var pila = ['Z'];            // el tope es el último elemento
    var salida = '';
    var traza = [];
    var estado = 0;

    while (estado !== ACEPTACION && estado !== SUMIDERO) {
      var simbEntrada, simbPila;
      switch (estado) {
        case 0:
          simbEntrada = cinta.charAt(i++);
          simbPila = LAMBDA;
          break;
        case 3:
          simbEntrada = cinta.charAt(i++);
          simbPila = pila.pop();
          break;
        default: // q1, q2, q4, q5, q7, q8
          simbEntrada = LAMBDA;
          simbPila = pila.pop();
      }

      var reaccion = null;
      if (estrategia === 2) {
        var r = indice(simbEntrada), p = indice(simbPila);
        if (r >= 0 && p >= 0) {
          var celda = TABLA_PDT_3D[estado][r][p];
          if (celda.nuevo_estado !== SUMIDERO) reaccion = [celda.nuevo_estado, celda.sec_push,
celda.sec_print];
        }
      } else {
        var pos = buscarEnTabla(estado, simbEntrada, simbPila);
        if (pos >= 0) reaccion = TABLA_PDT[pos].slice(3);
      }

      var paso = { estado: estado, read: simbEntrada, pop: simbPila };
      if (reaccion) {
        var print = reaccion[2] === 'letra' ? simbEntrada : reaccion[2];
        // push: el primer símbolo de la secuencia queda en el tope
        for (var k = reaccion[1].length - 1; k >= 0; k--) pila.push(reaccion[1].charAt(k));
        salida += print;
        paso.nuevo = reaccion[0];
        paso.push = reaccion[1];
        paso.print = print;
        estado = reaccion[0];
      } else {
        paso.nuevo = SUMIDERO;
        paso.push = LAMBDA;
        paso.print = LAMBDA;
        estado = SUMIDERO;
      }
      paso.pila = pila.slice().reverse().join('');
      paso.salida = salida;
      traza.push(paso);
    }

    return { aceptada: estado === ACEPTACION, posfija: estado === ACEPTACION ? salida : null, traza: traza
};
  }

  // ================================================================
  // PROBLEMA 3.2 - ANALIZADOR SLR(1) CON PILA DE CÁLCULO
  // ================================================================

  // GIC de la expresión posfija (reglas numeradas como en JFLAP):
  var REGLAS = [
```

```
  null,
  { izq: 'S', der: 'SS+' },  // r1
  { izq: 'S', der: 'SS-' },  // r2
  { izq: 'S', der: 'SS*' },  // r3
  { izq: 'S', der: 'SS/' },  // r4
  { izq: 'S', der: 'a' }     // r5
];

var COLUMNAS_LR = ['*', '+', '-', '/', 'a', '$', 'S'];

// Tabla LR obtenida con JFLAP: tabla[símbolo especial][símbolo]
var TABLA_LR = [
  { 'a': 's2', 'S': 1 },
  { 'a': 's2', '$': 'acc', 'S': 3 },
  { '*': 'r5', '+': 'r5', '-': 'r5', '/': 'r5', 'a': 'r5', '$': 'r5' },
  { '*': 's4', '+': 's5', '-': 's6', '/': 's7', 'a': 's2', 'S': 3 },
  { '*': 'r3', '+': 'r3', '-': 'r3', '/': 'r3', 'a': 'r3', '$': 'r3' },
  { '*': 'r1', '+': 'r1', '-': 'r1', '/': 'r1', 'a': 'r1', '$': 'r1' },
  { '*': 'r2', '+': 'r2', '-': 'r2', '/': 'r2', 'a': 'r2', '$': 'r2' },
  { '*': 'r4', '+': 'r4', '-': 'r4', '/': 'r4', 'a': 'r4', '$': 'r4' }
];

// Construye los dos arreglos unidimensionales a partir de la expresión posfija.
// Los operandos pueden ser números separados por blancos o letras con su valor.
function construirVectores(posfija, valores) {
  var simbolos = [], numeros = [];
  var tokens = posfija.trim().split(/\s+/).filter(Boolean);
  tokens.forEach(function (tok) {
    if (/^\d+(\.\d+)?$/.test(tok)) {
      simbolos.push('a');
      numeros.push(parseFloat(tok));
      return;
    }
    for (var k = 0; k < tok.length; k++) {
      var c = tok.charAt(k);
      if (OPERADORES.indexOf(c) >= 0) {
        simbolos.push(c);
      } else if (esLetra(c)) {
        if (!valores || valores[c] === undefined || valores[c] === '' || isNaN(valores[c])) {
          throw new Error('Falta el valor numérico del identificador "' + c + '".');
        }
        simbolos.push('a');
        numeros.push(parseFloat(valores[c]));
      } else {
        throw new Error('Símbolo no válido en la expresión posfija: "' + c + '".');
      }
    }
  });
  simbolos.push('$');
  return { simbolos: simbolos, numeros: numeros };
}

function operar(op, val1, val2) {
  switch (op) {
    case '+': return val1 + val2;
    case '-': return val1 - val2;
    case '*': return val1 * val2;
    case '/':
      if (val2 === 0) throw new Error('División por cero.');
      return val1 / val2;
  }
}

function evaluarPosfija(posfija, valores) {
  var traza = [];
  var v;
  try {
```

```
      v = construirVectores(posfija, valores);
    } catch (e) {
      return { aceptada: false, error: e.message, traza: traza };
    }
    var pilaAnalisis = [];
    var pilaCalculo = [];
    var iSimbolo = 0, iValor = 0;

    var simbEspecial = 0;
    pilaAnalisis.push(simbEspecial);
    var simbolo = v.simbolos[iSimbolo];
    var valorTabla = TABLA_LR[simbEspecial][simbolo];

    function registrar(accion) {
      traza.push({
        analisis: pilaAnalisis.join(' '),
        entrada: v.simbolos.slice(iSimbolo).join(''),
        accion: accion,
        calculo: pilaCalculo.map(formatear).join(' ')
      });
    }

    while (valorTabla !== 'acc') {
      if (valorTabla && valorTabla.charAt(0) === 's') {
        // desplazamiento
        registrar('Desplazar ' + valorTabla);
        pilaAnalisis.push(simbolo);
        simbEspecial = parseInt(valorTabla.substring(1), 10);
        pilaAnalisis.push(simbEspecial);
        simbolo = v.simbolos[++iSimbolo];
      } else if (valorTabla && valorTabla.charAt(0) === 'r') {
        // reducción
        var j = parseInt(valorTabla.substring(1), 10);
        var regla = REGLAS[j];
        var accion = 'Reducir ' + valorTabla + ': ' + regla.izq + ' → ' + regla.der;
        var resultado;
        try {
          if (regla.der === 'a') {
            resultado = v.numeros[iValor];
            accion += '  [apila ' + formatear(resultado) + ']';
          } else {
            var val2 = pilaCalculo[pilaCalculo.length - 1];
            var val1 = pilaCalculo[pilaCalculo.length - 2];
            resultado = operar(regla.der.charAt(2), val1, val2);
            accion += '  [' + formatear(val1) + ' ' + regla.der.charAt(2) + ' ' + formatear(val2) + ' = ' +
formatear(resultado) + ']';
          }
        } catch (e) {
          registrar(accion);
          return { aceptada: false, error: e.message, traza: traza };
        }
        registrar(accion);
        pilaAnalisis.splice(pilaAnalisis.length - 2 * regla.der.length);
        simbEspecial = pilaAnalisis[pilaAnalisis.length - 1];
        pilaAnalisis.push(regla.izq);
        simbEspecial = TABLA_LR[simbEspecial][regla.izq];
        pilaAnalisis.push(simbEspecial);
        // operar en pila de cálculo
        if (regla.der === 'a') {
          iValor++;
        } else {
          pilaCalculo.pop();
          pilaCalculo.pop();
        }
        pilaCalculo.push(resultado);
      } else {
        // casillero vacío
```

```
        registrar('Error');
        return { aceptada: false, error: 'Error sintáctico: no hay acción para (' + simbEspecial + ', ' +
simbolo + ').', traza: traza };
      }
      valorTabla = TABLA_LR[simbEspecial][simbolo];
    }
    if (simbolo !== '$') {
      return { aceptada: false, error: 'Quedan símbolos sin leer.', traza: traza };
    }
    registrar('Aceptar');
    var res = pilaCalculo.pop(); // vaciar pila: único valor
    return { aceptada: true, resultado: res, traza: traza };
  }

  function formatear(n) {
    return Number.isInteger(n) ? String(n) : String(Math.round(n * 1e6) / 1e6);
  }

  function nombreEstado(e) { return 'q' + e; }

  var api = {
    LAMBDA: LAMBDA, FIN: FIN, SUMIDERO: SUMIDERO,
    TABLA_PDT: TABLA_PDT, TABLA_PDT_3D: TABLA_PDT_3D, INDICE_SIMBOLO: INDICE_SIMBOLO,
    REGLAS: REGLAS, TABLA_LR: TABLA_LR, COLUMNAS_LR: COLUMNAS_LR,
    convertirInfijaAPosfija: convertirInfijaAPosfija,
    evaluarPosfija: evaluarPosfija,
    formatear: formatear, nombreEstado: nombreEstado
  };
  if (typeof module !== 'undefined' && module.exports) module.exports = api;
  else global.TP3 = api;
})(this);
```